

# Phase 2: time trial, timing and results

BeamMP Race Manager    Client: dhardesty99

The core of the whole product. Records, XP, teams and challenges all sit on top of what gets built here, so anything wrong in this phase is wrong in every phase after it.

Two things decide whether this phase is any good: whether a lap time is honest, and whether the server and the network are still quiet while it is being measured. Most of this document is about those two.

## 1 What gets delivered

From his document, unchanged:

- Time Trial, with a lap count if he wants laps
- Controller or Wheel selected before the run
- Teleport to the start, and the run arms when he drives through the trigger
- End Race stops all timing at once
- A split recorded at every checkpoint
- Missed checkpoints caught and penalised
- The full results screen
- No live leaderboard during the run

Plus two things he asked for after the plan was written:

- **Penalties only apply during a race.** Free driving costs nothing. That decides the shape of the race state machine, so it belongs here rather than in phase 3.
- **His colour scheme.** Dark red, black and grey, with a little orange and yellow for contrast. The results screen is the first thing worth styling.

## 2 How a split is measured

This is the decision the whole phase hangs on, and the obvious approach is wrong.

The obvious approach is: the client says it crossed a gate, and the server stamps the time it hears about it. That is simple and it is unusable. The message takes half the round trip to arrive, so a driver on 200 ms ping has roughly 100 ms added to every split, and jitter means it is a different amount every time. On a 29 gate course that is seconds of error, and it punishes the player with the worse connection. He is in the United States and I am in Pakistan; we would never agree on a lap time.

So the crossing is stamped by the client, at the exact frame the trigger fires, and the server converts that stamp into its own clock and then decides whether to believe it.

1. The engine fires `onBeamNGTrigger`. The client reads its own monotonic clock at that moment and sends the gate index with it.
2. The server converts the client stamp into server time using an offset it maintains per player.

3. The server validates the result. If it does not survive, the arrival stamp is used instead and the run is flagged.

The client still cannot cheat, because it never decides anything. It offers a number and the server decides whether that number is possible.

### 3 Clock offset

Standard four timestamp exchange, the same arithmetic NTP uses. The server sends a probe carrying  $T_1$ ; the client notes  $T_2$  on receipt and replies immediately carrying  $T_2$  and its send time  $T_3$ ; the server notes  $T_4$  on arrival.

$$\begin{aligned}\text{delay} &= (T_4 - T_1) - (T_3 - T_2) \\ \text{offset} &= \frac{(T_2 - T_1) + (T_3 - T_4)}{2}\end{aligned}$$

Two details matter more than the formula:

**Keep the sample with the lowest delay, not the average.** A high delay sample is one that sat in a queue, and queuing is asymmetric, which is exactly what poisons the offset. NTP keeps the best of the last eight for this reason. We keep the best of the last eight probes over the preceding minute.

**The probe rides the existing bus.** It is two extra fields on a message that is already being sent every tick a player has anything queued. No new message, no new timer, no added traffic.

### 4 Validating a claimed crossing

A stamp from the client is a claim. Four checks, cheapest first, and all of them are arithmetic on numbers already in memory:

1. **Monotonic.** The corrected time must be later than the previous split on this run. Times do not go backwards.
2. **Not in the future.** It must not be later than the server clock now.
3. **Inside the trust window.** It must not be earlier than now  $-(\text{delay} + \text{margin})$ . A client cannot have crossed a gate long before we heard about it.
4. **Physically possible.** We captured the coordinates of every gate, so we know the distance from gate  $n$  to gate  $n+1$ . If the claimed time between them implies a speed above the ceiling, it is rejected.

That last one is the useful one, and it is free: the distance between consecutive gates is computed once when the course is loaded, not per crossing.

A claim that fails is not a kick. The arrival stamp is used, the run is marked, and the reason is recorded. Phase 5 decides whether a marked run can hold a record. Nobody gets thrown off the server because their connection hiccuped.

### 5 Race state machine

One race state per player for time trial. Phase 4 turns this into a shared grid, so the shape is chosen now to survive that.

idle → armed → running → finished,    or running → abandoned

| From     | To        | On                                        |
|----------|-----------|-------------------------------------------|
| idle     | armed     | picks a course, teleports to the start    |
| armed    | running   | drives through the start gate             |
| running  | running   | crosses a gate, split recorded            |
| running  | finished  | crosses the finish on the final lap       |
| running  | abandoned | End Race, disconnect, or leaves the level |
| finished | idle      | closes the results                        |

**Penalties only exist in running.** That is his change, and it falls out of the machine rather than being bolted onto it. It also means the bottom bar in phase 3 has one question to ask, not a special case per button.

## 6 Splits, laps and allocation

A run holds the course and mode, the lap count and current lap, start and finish stamps, a split per gate per lap, the missed gates, and the penalties with their reasons.

Splits are seconds from the start of the run, not deltas. Deltas are a presentation choice and can always be computed; a stored delta cannot be recovered into an absolute once a gate is missed.

**The splits array is allocated once, when the run is armed.** The gate count and the lap count are both known at that moment, so the whole table is built to its final size and written into by index. Not one table is created while a car is on track.

This is the phase 1 rule carried forward and it is the one that matters most here. Allocating per crossing would create garbage, the collector would run, and the game would pause for a few milliseconds. That pause is exactly the tick lag he complained about in other race mods, and it would land during a lap.

## 7 Missed checkpoints

A gate is missed when a later gate fires while an earlier one has no split for this lap. That catches both a deliberate cut and a car tunnelling a gate at speed.

The penalty is per missed gate, and the reason carries the gate number, so the breakdown in the results reads as something a person can argue with.

Circuit courses wrap, so gate 1 firing after gate 29 is a lap, not a miss. The lap boundary is the start gate, which is why the start line is stored separately from the checkpoint list.

## 8 Results screen

Only after everyone has finished. No live leaderboard, which was his call and removes the largest network cost in the system.

| Panel          | Notes                                         |
|----------------|-----------------------------------------------|
| Corrected time | penalties included. This decides the result.  |
| Click a name   | that player's penalties, each with its reason |
| Dirty time     | clean time, information only, decides nothing |

| Panel                 | Notes                       |
|-----------------------|-----------------------------|
| Overall best lap      |                             |
| Personal best lap     | click to expand every lap   |
| Diff to the car ahead |                             |
| Diff to the leader    |                             |
| Personal best split   | click to expand every split |

The whole payload is built once, when the last driver finishes, and sent once. It is never recomputed on a timer.

## 9 Performance budget

Measured on his box, idle, with phase 1 running:

| Resource           | In use                        | Limit            |
|--------------------|-------------------------------|------------------|
| Server CPU         | 0.7%                          | 200% (two cores) |
| Server memory      | 11 MB                         | 8192 MB          |
| Plugin lua state   | 206 KB                        | shares the above |
| Network, ten hours | 15 MB total                   | unmetered        |
| Client frame rate  | 128 avg, 108 one per cent low | baseline to hold |

There is an enormous amount of headroom. The risk is not running out of it, it is writing something that scales badly and only shows up with ten cars on a 29 gate course. So the budget is set as rules rather than as a ceiling.

### The server gains no new per tick work

Phase 1 has one 100 ms timer. Phase 2 adds nothing to it. Every piece of timing is event driven: a split is computed when a gate fires and never again, and a run is finalised once when the last driver crosses.

Anything that would need a timer is a design mistake. There is no live leaderboard to recompute, no gap to recalculate, no position to poll.

### The traffic is small and it is bounded

A split message is a gate index and a number, roughly 40 bytes of JSON. It is queued on the bus that already exists and goes out in the batch that is already being sent for that tick.

A full grid of ten drivers on a 29 gate circuit generates 290 split messages per lap. A Baja lap is minutes long, so that is under one message per second across the whole server, and multiple splits landing in the same tick coalesce into one batch.

For comparison, BeamMP is synchronising vehicle position for every car many times a second on the same channel. Our timing traffic is a rounding error against it, and that is the point: the channel is shared, so anything we add competes with car positions, and the way to not make everyone look laggy is to add almost nothing.

The clock probe adds no messages at all. It is two fields on a message that was already going.

## Memory is bounded before the run starts

A run of ten laps on 29 gates holds 290 numbers. Ten drivers is 2900 numbers, call it 100 KB with table overhead against a lua state currently using 206 KB and a container limit of 8 GB.

The number is not the point. The point is that it is allocated once at arm time and never grows, so there is nothing for the collector to do mid race.

## Disk is written once

On race end, and on the existing slow timer. Never per split. A split that reaches disk before the run is over is a bug.

## The client draws, it does not compute

The running clock on screen is the client counting locally from a start time the server gave it. It is not the server pushing a time ten times a second.

That keeps the phase 1 rule intact: the interface redraws when a number changes, at most ten times a second, and nothing in the mod does per frame work unless a tool is open.

## 10 Acceptance criteria

Numbers, not opinions. Measured on his server before the phase is called done:

- Client frame rate within **3 average and 5 one per cent low** of the phase 1 baseline of 128 and 108, same spot, same car, nothing recording.
- Server CPU under **5%** with a full grid mid race.
- Plugin lua state under **1 MB** with a full grid mid race.
- Added network under **2 KB per driver per lap**.
- Split accuracy within **30 ms** of a locally timed reference run, taken on a connection with 200 ms ping.
- Zero handler errors in the log across a full test session.

If frame rate misses that bar, it gets fixed before delivery rather than noted in the handover. That was promised in writing and it stands.

## 11 Load test

The honest version needs bodies, and he has one account. Failing that:

1. Spawn AI traffic to put cars on the map and measure with those.
2. Drive a course with the gate count of Baja 1000 rather than a five gate test loop, since gate count is the thing that scales.
3. Replay a run against the mock host with the clock driven by hand, ten drivers at once, and watch the lua state size and the message count.

The third one catches scaling faults without needing a single other human, and it is the one to build first.

## 12 Testing

The mock host gets a clock that can be driven by hand, so a run plays through in a test without waiting for real seconds. Cases worth having before the code is written:

- A clean lap. Splits in order, corrected equals dirty.
- A missed gate. Penalty logged against the right gate number.
- A gate crossed twice. The second crossing ignored.
- Gates crossed out of order.
- Disconnect mid run. Run abandoned, nothing half written to disk.
- End Race mid run. All timing stops at once.
- A claimed crossing earlier than the previous split.
- A claimed crossing in the future.
- A claimed crossing implying an impossible speed between two gates.
- A clock offset that jumps, then settles.
- Two runs on one course by one player. Personal best updates only when it should.
- Ten simulated drivers on a 29 gate circuit. Allocation does not grow during the run.

Then on his server, with two drivers, which is the part that is still blocked.

## 13 Protocol additions

| Direction        | Channel                    | Carries                                              |
|------------------|----------------------------|------------------------------------------------------|
| client to server | <code>race.arm</code>      | course, mode, lap count                              |
| client to server | <code>race.end</code>      | nothing                                              |
| client to server | <code>cp.hit</code>        | gate index and the client stamp at the crossing      |
| client to server | <code>clock.pong</code>    | the two client timestamps, rides an existing message |
| server to client | <code>race.state</code>    | the state machine, on change only                    |
| server to client | <code>race.split</code>    | the split just recorded, for the HUD                 |
| server to client | <code>race.result</code>   | the whole results payload, once                      |
| server to client | <code>race.teleport</code> | where to put the car when arming                     |
| server to client | <code>clock.ping</code>    | the server stamp, rides an existing message          |

## 14 What is needed from him

| Item            | Needed for                     | Status                      |
|-----------------|--------------------------------|-----------------------------|
| A second driver | two driver testing             | in hand                     |
| Captured routes | something real to time         | tool is his, waiting on him |
| Race classes    | phase 5, shapes the record now | waiting                     |
| Logo files      | results screen styling         | sent, colours noted         |

A five gate test loop does not exercise this phase. Gate count is what scales, so the sooner Baja 1000 is captured the sooner the timing is being tested against something real.

## 15 Order of work

1. Race state machine, server side, with tests
2. Clock offset, and the validation checks that depend on it
3. Splits and lap counting, allocated once at arm time
4. Missed checkpoint detection
5. Arming, teleport, and the End Race button
6. Ten driver replay against the mock host, watching allocation
7. Results payload
8. Results screen, in his colours
9. Frame rate measured against the baseline

Steps 1 to 4 are all server side and all testable without the game, which is where the risk is and where the mock host earns its keep. Nothing goes near the interface until the timing is proven against a clock we control.

The CoPilot camera gets a quiet look during this phase. It is the least known part of the whole job, and week eleven is the wrong time to find out it is hard.